

CRM Architecture Complete

Architecture technique globale, frontend, backend, base de données, API, sécurité, DevOps

Méthodologie : analyse exhaustive du code source réel (backend Node.js/Express/Sequelize, frontend Flutter), aucune fonctionnalité supposée

Légende : **EXISTE** confirmé dans le code — **PARTIEL** implémenté partiellement — **NON IDENTIFIÉ** absent du code analysé — **RECOMMANDATION** architecture cible, non existante

Date de génération : 29 août 2026

Sommaire

1. Résumé exécutif	
2. Présentation du CRM	
3. Cartographie du repository	
4. Architecture globale	
5. Technologies utilisées	
6. Architecture Frontend	
7. Architecture Backend	
8. Architecture Base de données (vue d'ensemble)	
9. Architecture API	
10. Authentification	
11. MFA	
12. Autorisation / RBAC	
13. Gestion des erreurs	
14. Email	
15. Notifications	
16. Upload / Documents	
17. Cron / Scheduler	
18. Déploiement, DevOps, PM2, Environnements	
19. Logging / Monitoring	
20. Responsive Web / Tablet / Mobile	
21. Sécurité	
22. Points forts	
23. Problèmes techniques identifiés	
24. Architecture cible (recommandations)	
25. Diagrammes complémentaires	
26. Conclusion	

1. Résumé exécutif

Objectif du CRM : le code analysé montre un CRM d'entreprise pour CBI Tunisia / Probar Distribution, centré sur la gestion de projets immobiliers/industriels (pipeline commercial, devis, bons de commande), les contacts commerciaux et leur relance, les ressources humaines (congés/sorties), la maintenance (interne et industrielle), le suivi de production industrielle (POR PROMESH, PROBAR, Mélange) et un Copilot IA conversationnel. Aucun module Finance transactionnel (factures, paiements, bons d'achat) ni module OCR n'a été identifié dans le code source analysé.

Utilisateurs : 7 rôles réels confirmés dans l'ENUM Sequelizeize (`users.role`) — `user`, `commercial`, `accueil`, `admin`, `superadmin`, `superadmin2`, `responsable_logistique_achat` — complétés par des autorisations câblées en dur par adresse email (compte root admin, gestionnaire RH, gestionnaire maintenance) plutôt qu'un système de permissions génériques.

Architecture : backend Node.js/Express 5 + Sequelize/PostgreSQL, organisé en **trois strates architecturales coexistantes** (routes monolithiques historiques, routes+controller simples, modules en couches avec validation Joi) — pas une convention unique. Frontend Flutter multiplateforme (Android/iOS/Web/Windows/macOS/Linux), routing GoRouter, état géré principalement par GetX (~130 fichiers) avec Provider en périphérie (3 fichiers, auth globale + langue).

Base de données : PostgreSQL, ~44 modèles Sequelizeize, migrations exclusivement via `sequelize-cli` (aucun `sync()` automatique en production, confirmé par un commentaire du code lui-même).

IA : un Copilot IA conversationnel complet (19 agents, 76 outils, RAG documentaire, protocole MCP interne) fait l'objet du **Document 3** dédié — non détaillé ici au-delà de sa place dans l'architecture globale.

Sécurité : authentification JWT à deux secrets (access/refresh), RBAC multi-niveaux (middleware global + middlewares spécialisés par domaine), mais plusieurs constats techniques réels identifiés — un système MFA complet mais PARTIEL (jamais déclenché par la connexion réelle), un second mécanisme JWT parallèle non unifié, une fuite de stack trace confirmée dans une réponse d'API.

Flux métier principaux : Login → (MFA non déclenché en pratique) → Dashboard → Module métier → Action → Validation → Résultat, détaillés dans le **Document 2** (architecture métier).

2. Présentation du CRM

Le dépôt contient deux applications couplées :

Application	Dossier	Stack	Rôle
Backend	Backend Master/	Node.js, Express 5, Sequelize 6, PostgreSQL	API REST + WebSocket (Socket.IO), logique métier, base de données, kernel IA
Frontend	Dash Master Toolkit/	Flutter (SDK ^3.6.0), GoRouter,GetX/Provider	Application multiplateforme (web, desktop, mobile)

Le nom des modèles et modules (Company, Architect, Engineer, Client, Project avec champs `productFamily` ENUM `PROBAR/PROMESH`) indique un CRM de **gestion de projets et de distribution industrielle** (matériaux de construction/industriels), avec un sous-système de suivi de production en atelier (POR PROMESH, PROBAR, Mélange, maintenance de machines) distinct du CRM commercial/projets.

Documentation existante identifiée dans le dépôt

Un ensemble de documents d'audit internes existe déjà sous `Backend Master/docs/` (`audit/` , `security/` , `performance/` , `ai/` , `user-guide/` , `admin-guide/`) — produit par des passes d'audit antérieures de ce même dépôt. Ces documents sont utilisés dans le présent rapport comme source secondaire, systématiquement recoupée avec une lecture directe du code : lorsqu'un écart a été trouvé (ex. l'affirmation « aucun fournisseur LLM n'est branché », contredite par la configuration réelle `AI_LLM_PROVIDER=ollama` du `.env` actuel), l'écart est signalé explicitement plutôt que silencieusement corrigé.

3. Cartographie du repository

Repository (source code/)

```
├─ Backend Master/                                - API Node.js/Express + kernel IA
├─   └─ src/
├─     └─ app.js                                  - point d'entree, montage des middlewares/routes
├─     └─ db.js                                   - connexion Sequelize/PostgreSQL
├─     └─ routes/                                - routes "legacy" (logique inline, sans service dedie)
├─     └─ controllers/                           - controllers simples (architect, company, engineer...)
├─     └─ modules/                               - architecture en couches (route/controller/service/repository/validator)
├─       └─ projects/ pipeline/ commercial-contacts/ hr-requests/
├─       └─ maintenance-requests/ industrial-records/ recuperables/
├─       └─ por-promesh/ project-actions/ project-activities/
├─       └─ archive-requests/ kanban/ dashboard/ admin-dashboard/
├─       └─ ai-copilot/ ai-dashboard/             (kernel IA, Document 3)
├─     └─ models/                                - ~44 modeles Sequelize
├─     └─ migrations/                            - migrations sequelize-cli (source de verite du schema)
├─     └─ seeders/                               - jeux de donnees initiaux (pipeline, superadmin)
├─     └─ services/                             - services transverses (email, MFA, calendrier Google, notifications)
├─     └─ middleware/                           - 17 fichiers (auth, RBAC, upload, validation)
├─     └─ cron/                                 - 5 jobs planifies (node-cron)
├─     └─ config/                               - constantes de permission (emails admin en dur)
├─     └─ ai/                                   - kernel Copilot IA (Document 3)
├─     └─ utils/                                - JWT, logger, mailer
├─     └─ uploads/                              - stockage fichiers local (disque)
├─   └─ docs/                                   - documentation d'audit preexistante
├─   └─ test/                                   - suites Jest
├─   └─ package.json
├─ Dash Master Toolkit/                          - application Flutter
├─   └─ lib/
├─     └─ main.dart                             - point d'entree
├─     └─ route/my_route.dart                   - GoRouter (1312 lignes, toutes les routes)
├─     └─ app_shell_route/                      - layout persistant (sidebar/topbar/shell)
├─     └─ providers/                            - AuthService, ApiClient (Dio), stockage
├─     └─ services/                             - clients API par domaine metier
├─     └─ forms/                               - coeur metier (POR PROMESH, PROBAR, RH, recuperables...)
├─     └─ dashboard/                           - tableaux de bord par verticale
├─     └─ application/                         - kanban, chat, calendrier, utilisateurs
├─     └─ pages/                               - auth, projets (legacy), pages simples
├─     └─ ai/                                  - Copilot IA frontend (Document 3)
├─     └─ models/ tables/ others/              - DTO, ecrans generiques/demo du template
├─     └─ theme/ localization/ constant/ core/ utils/ widgets/
├─   └─ android/ ios/ web/ windows/ macos/ linux/ - projets natifs complets (6 plateformes)
├─   └─ test/                                  - 1 seul fichier de test identifie
├─   └─ pubspec.yaml
├─ .github/workflows/backend-ci.yml             - CI GitHub Actions (backend uniquement)
├─ AI-Business-Copilot/                         - journaux de missions IA (documentation historique)
├─ Documentation/                              - site de documentation statique preexistant
```

Dossier	Rôle	Dépendances
src/routes/	Routes historiques, logique métier inline	Modèles Sequelize directement, services ponctuels
src/modules/*	Architecture en couches la plus récente	Route → Controller → Service → Repository, validation Joi
src/ai/	Kernel Copilot IA, Ports & Adapters	Isolé du reste du CRM sauf via le Bridge (src/modules/ai-copilot/)
lib/ai/	Frontend Copilot IA	GetX, Dio partagé (ApiClient)
lib/forms/	Cœur métier PROBAR (industrie, RH, récupérables)	GetX, services API par domaine

4. Architecture globale

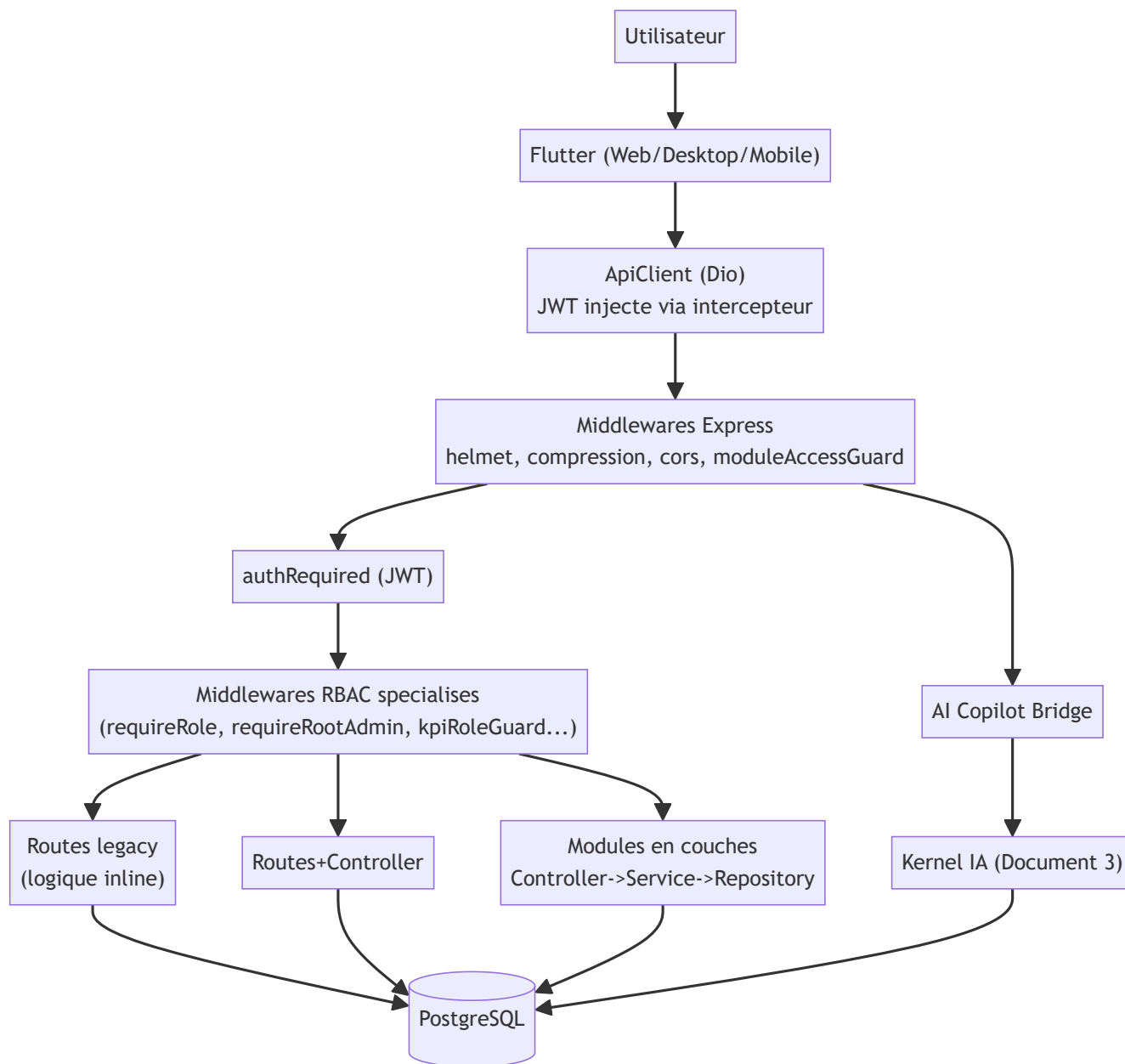


Figure 1 — Architecture globale réelle (adaptée : pas de couche "Controller" unique, trois styles de routage coexistent)

Contrairement à un schéma HTTP→Router→Controller→Service→Repository→DB uniforme, le code réel présente **trois chemins distincts** selon le module (détaillé section 7). L'unique point d'unification transversal est le middleware `moduleAccessGuard`, monté globalement avant toute route (`app.js:126`).

5. Technologies utilisées

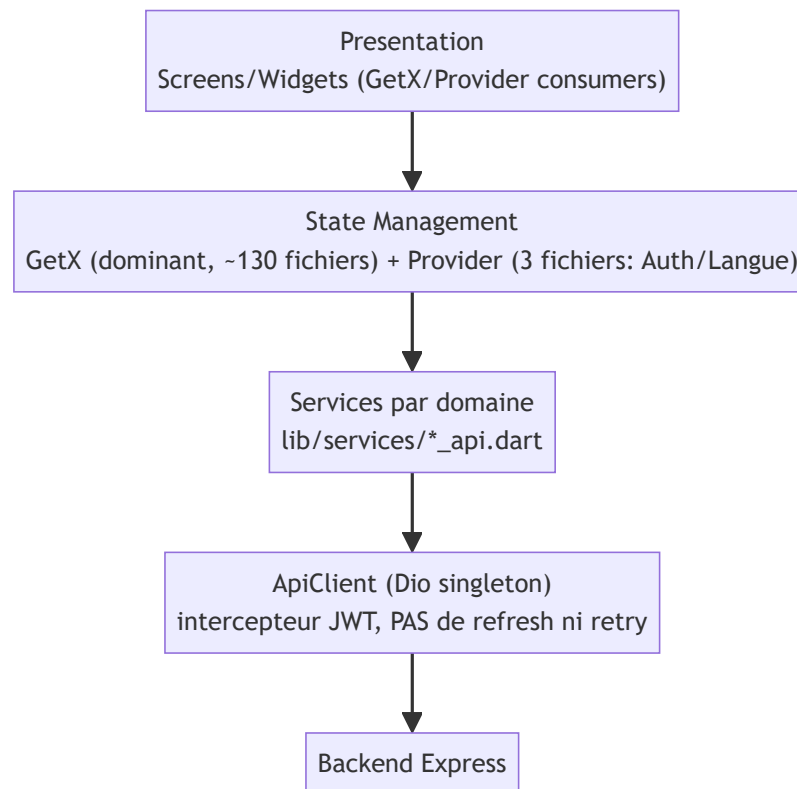
Backend (Backend Master/package.json)

Catégorie	Bibliothèques réelles
Framework HTTP	express ^5.2.1
ORM / DB	sequelize ^6.37.8, pg, pg-hstore, sequelize-cli
Authentification	jsonwebtoken ^9.0.3, bcrypt ^6.0.0
Sécurité HTTP	helmet ^8.1.0, cors ^2.8.5, cookie-parser, express-rate-limit ^8.2.1
Validation	joi ^18.2.3 (modules récents), zod ^4.4.3 (kernel IA uniquement)
Fichiers	multer ^2.1.1, exceljs, xlsx, csv-parser, pdfkit, pdf2json
Email	nodemailer ^7.0.13
Temps réel	socket.io ^4.8.1
Planification	node-cron ^4.2.1
IA	@langchain/core (kernel IA, Document 3)
Autres	redis ^4.7.1 (déclaré, usage hors module IA non identifié), twilio ^6.0.2 (WhatsApp), geoip-lite, ua-parser-js, dayjs, uuid, axios, dotenv
Tests	jest ^30.4.2, supertest, nodemon (dev)

Frontend (Dash Master Toolkit/pubspec.yaml)

Catégorie	Bibliothèques réelles
Routing	go_router ^14.8.0
État	get ^4.6.5 (dominant), get_storage ^2.0.3, provider ^6.1.2 (périphérique)
Réseau	dio ^5.7.0, http ^1.2.2, socket_io_client ^2.0.3+1
Sécurité	flutter_secure_storage ^9.2.2, jwt_decoder ^2.0.1
UI/Charts	fl_chart, syncfusion_flutter_charts/maps/calendar ^29.x, responsive_framework ^1.5.1
Cartographie	google_maps_flutter, flutter_map, geolocator, geocoding
Fichiers/Export	file_picker, excel, csv, pdf, printing, image_picker
Sérialisation	Aucune (pas de json_serializable/freeze) — fromJson/toJson 100% manuels

6. Architecture Frontend



Point d'entrée et bootstrap

`main.dart` : `GetStorage.init()` → `AuthService().restoreSession()` (session locale valable 7 jours, indépendante de l'expiration réelle du JWT) → `MultiProvider` (`AppLanguageProvider`, `AuthService`) → `MaterialApp.router` avec `responsive_framework` (5 breakpoints XS→XL) et `ThemeController` (GetX).

Routing (GoRouter, `lib/route/my_route.dart`, 1312 lignes)

Un `redirect` global décide de la destination selon `AuthService().isLoggedIn` et le rôle : `responsable_logistique_achat` → dashboard POR PROMESH ; `commercial` → KPI contacts commerciaux ; autres rôles → dashboard général. **Partiellement propre** : des `print()` de diagnostic marqués « DIAGNOSTIC TEMPORAIRE » subsistent dans le code de production.

Modules routés hors `ShellRoute` : authentification (login/signup/reset), sélection commercial. Modules routés sous `ShellRoute` (layout persistant) : Dashboard, Projets, Contacts commerciaux, Devis/Bons de commande, Kanban, Pipeline, Calendrier, Chat, Archive Requests (réservé root admin), Maintenance Requests, RH, POR PROMESH, Production (PROBAR/PROMESH/Mélange/Maintenance industrielle), Récupérables, Administration Utilisateurs, écrans génériques du template (tables/composants démo).

Gestion d'état

Confirmé : GetX (`package:get`) est le pattern **dominant** (~130 fichiers hors module IA) — contrôleurs métier, singletons (thème), navigation. Provider (`package:provider`) est utilisé en périphérie, uniquement pour `AppLanguageProvider` et `AuthService` exposés à la racine de l'app (3 fichiers). Ni Riverpod ni Bloc ne sont utilisés. *Ce constat corrige une affirmation de la documentation d'audit préexistante (`Architecture_Report.md`), qui présentait Provider comme le choix de gestion d'état du projet — la lecture directe du code montre que GetX est en réalité prépondérant.*

ApiClient et AuthService

`ApiClient` (Dio singleton) : timeout 15s, intercepteur unique injectant `Authorization: Bearer` depuis `GetStorage`. **Partiellement implémenté** : sur une erreur 401, le stockage local est nettoyé (`clearAuth()`) mais aucun refresh automatique du token, aucune re-tentative de requête, et aucune déconnexion "active" (pas de `notifyListeners()` / navigation forcée immédiate) ne sont déclenchés. **Aucun intercepteur de retry n'existe.**

`AuthService` (singleton, `ChangeNotifier`) : double stockage du token (`flutter_secure_storage` comme source de vérité + miroir en clair dans `GetStorage` consommé par l'intercepteur Dio et lu directement à plusieurs endroits, ex. `my_route.dart:624-626`). **Aucune méthode de**

refresh token côté frontend n'a été trouvée — la session "dure" via une expiration locale fixe de 7 jours, pas via un appel à `/auth/refresh` .

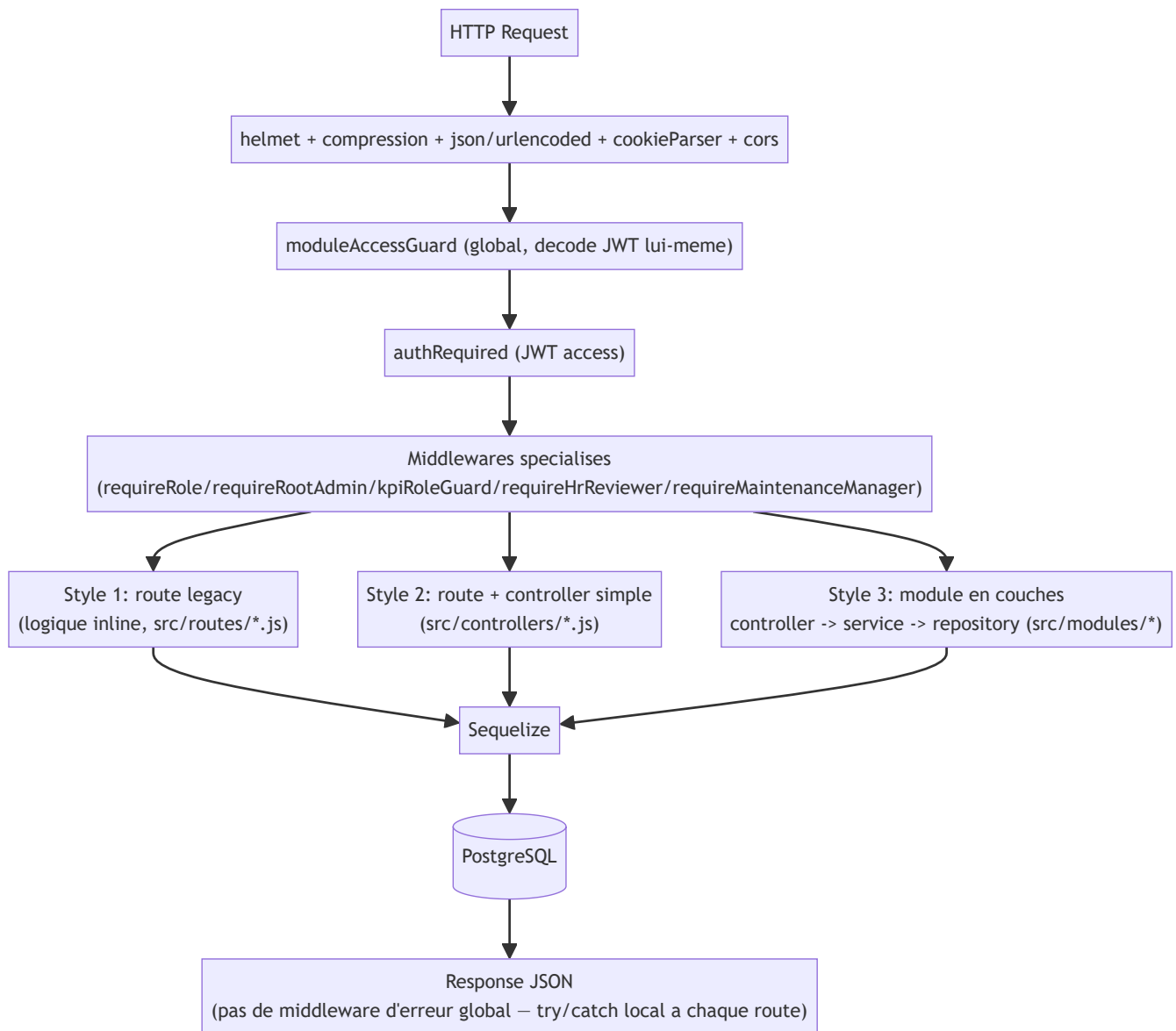
Thème, i18n, structure

Thème clair/sombre complet (`Styles.lightTheme/darkTheme`), police unique InterTight. Deux contrôleurs de thème coexistent (`ThemeController` , réellement utilisé ; `AppThemeController` , non référencé ailleurs — code mort probable). Internationalisation réelle : 7 fichiers de traduction JSON (ar/bn/en/fr/hi/id/ru) avec support RTL, mais l'essentiel des écrans métier ajoutés pour PROBAR (formulaires industriels) semble rédigé en dur en français plutôt que via le mécanisme `AppLocalizations.translate()` .

Responsive

`responsive_framework` (5 breakpoints) pilote la bascule sidebar fixe/ `Drawer` dans `AppShell` ; 86 fichiers utilisent en complément `MediaQuery` / `LayoutBuilder` ponctuellement — détail section 20.

7. Architecture Backend



Point d'entrée — `src/app.js`

Ordre exact des middlewares globaux : `helmet()` → `compression()` → `express.json({limit:"10mb"})` → `express.urlencoded` → `cookieParser()` → `cors({origins whitelistées en dur})` → `moduleAccessGuard`. Aucun rate-limiting global (seulement sur des routes spécifiques d'auth). `trust proxy` explicitement désactivé, avec un commentaire du code documentant lui-même le risque en cas de déploiement derrière un reverse-proxy.

Point technique notable : les préfixes `/projects`, `/utils` et `/commercial-contacts` sont chacun **montés deux fois** avec deux routers Express différents (module en couches en premier, puis router legacy) — la résolution effective dépend de l'ordre interne des routes de chaque router, source de confusion pour la maintenance.

Trois strates architecturales coexistantes

Strate	Pattern	Exemples réels
1. Legacy	Logique complète inline dans la route (validation, requêtes SQL, agrégations)	<code>routes/projects.routes.js</code> (4330 lignes), <code>auth.routes.js</code> (643 lignes), <code>commercial_contacts.routes.js</code> (1211 lignes)
2. Route + Controller simple	Route déléguant à un controller dédié, sans repository	<code>architect.routes.js</code> → <code>architect.controller.js</code> , idem <code>company/engineer/projectAction/projectTimeline</code>

3. Module en couches	Route → Controller → Service → Repository, validation Joi dédiée	src/modules/projects/ , pipeline/ , hr-requests/ , por-promesh/ , recuperables/ , industrial-records/
----------------------	--	---

Validation des entrées

Coexistence de Joi (modules récents, un fichier *.validator.js dédié par module) et de validation manuelle inline (fonctions locales dans les routes legacy : validatePayload , isValidEmail , isStrongPassword ...). express-validator : non identifié . Zod n'est utilisé que par le kernel IA.

Cron / Scheduler (src/cron/ , node-cron)

Job	Fréquence	Rôle	Garde-fou
checkProjects.js	Toutes les heures	Archive les projets sans ingénieur après échéance	AUTO_EMAIL_JOBS_ENABLED + AUTO_ARCHIVE_ENABLED
projectCron.js	Quotidien 8h	Détecte champs manquants, relance email, archivage	AUTO_EMAIL_JOBS_ENABLED
followup.job.js	Chaque minute (marqué « TEST » dans le code)	Rappels de relance/action (email+notification+WhatsApp best-effort)	AUTO_EMAIL_JOBS_ENABLED + déduplication (suite à un incident réel, voir section 23)
googleCalendarChannelRenewal.job.js	Quotidien 3h	Renouvelle les canaux push Google Calendar	AUTO_EMAIL_JOBS_ENABLED
proactiveAiMonitor.job.js	Non précisé dans cet audit (module IA)	Surveillance proactive IA (Document 3)	AI_PROACTIVE_MONITORING_ENABLED

8. Architecture Base de données (vue d'ensemble)

L'ERD complet, le détail colonne par colonne et la chronologie des migrations font l'objet du **Document 4 (CRM_Database_API_Architecture.pdf)**. Cette section donne une vue d'ensemble structurée.

Domaine	Modèles principaux
Identité / Auth	User, UserProfile, TrustedDevice, MfaOtp, MfaAttemptLog, PasswordResetLog
Projets / Pipeline	Project, ProjectAction, ProjectActionType, ProjectActivity, ProjectComment, ProjectDevis, ProjectBonDeCommande, ProjectReminder, ProjectMember, UserProject, PipelineStage
Commercial	CommercialContact, CommercialContactAction, CommercialContactProduct, CommercialContactRelance, CommercialContactReminder, CommercialContactStatusHistory, CommercialProject
RH	HrRequest, HrRequestActivity
Maintenance (interne)	MaintenanceRequest, MaintenanceRequestActivity, MaintenanceRequestComment, MaintenanceActivity
Production industrielle	PorPromesh (+ 6 tables liées : ArretMachine, Attachment, Consommation, ControleQualite, NonConformite, ProcessControl), IndustrialRecord (table générique partagée PROBAR/Mélange/Maintenance industrielle, discriminée par colonne <code>module</code>)
Récupérables	RecuperableFiche, RecuperableLigne
Archive	ArchiveRequest, ArchiveRequestMessage
Calendrier	Task, CalendarEventSync, GoogleCalendarAccount
Notifications / Email	Notification, EmailQueue
Tiers	Company, Architect, Engineer, client.model.js

Pattern architectural notable : `IndustrialRecord` (table `industrial_records`) est une **table générique unique** partagée par 3 modules fonctionnels distincts (PROBAR, Mélange, Maintenance industrielle), discriminée par une colonne ENUM `module` — confirmé littéralement par le commentaire du modèle lui-même : « Table générique partagée par les modules PROBAR, MÉLANGE et MAINTENANCE (pas de schéma métier dédié par module) ». Le champ `melangeData` (JSONB) stocke les données spécifiques au module Mélange sans schéma structuré. Ce choix diffère de POR PROMESH, qui dispose d'un schéma normalisé propre (7 tables liées).

Migrations : gérées exclusivement via `sequelize-cli` (`src/migrations/`), jamais de `sequelize.sync()` en production — confirmé par un commentaire explicite du code (`app.js`).

9. Architecture API

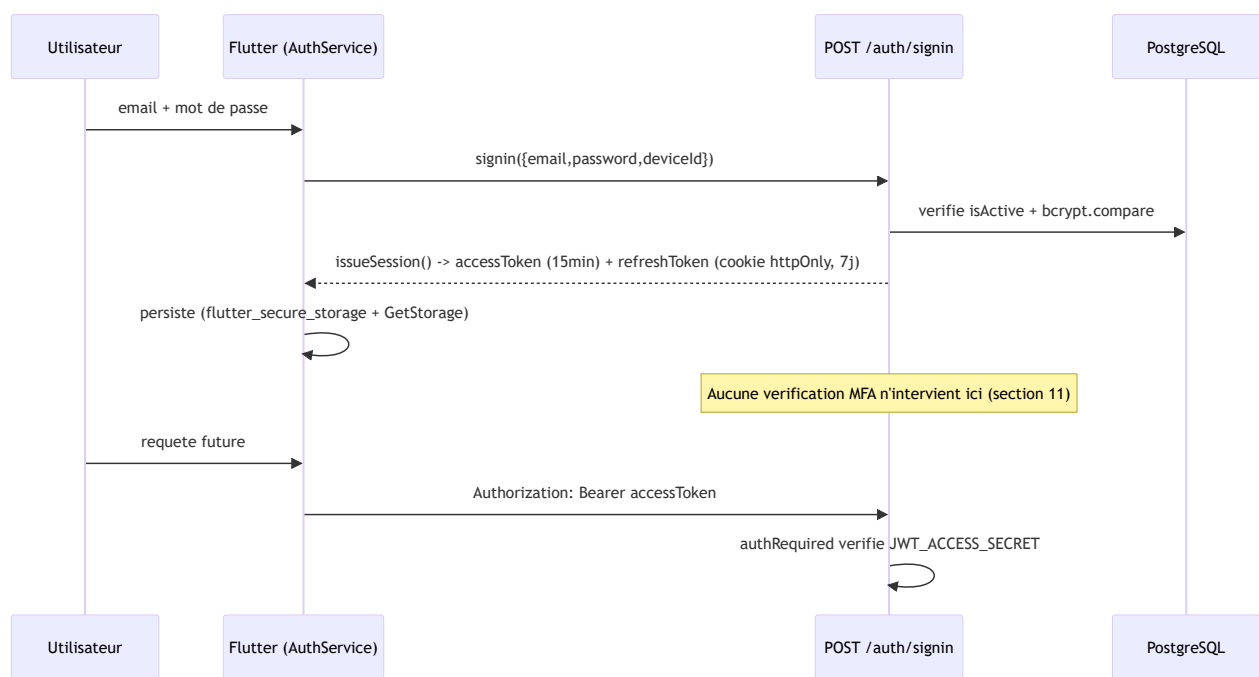
Le catalogue complet des endpoints fait l'objet du Document 4. Cette section décrit le pattern architectural général.

Le préfixage de route est déclaré dans `app.js` par une longue liste de `app.use("/prefix", router)`, avec les modules en couches montés en premier (pipeline-stages, dashboard, archive-requests, maintenance-requests, crm, por-promesh, industrial-records, hr-requests, recuperables, admin-dashboard, ai-dashboard, ai-copilot), suivis des routes legacy (projects — second montage, auth, admin, tasks, notifications, users, uploads statiques, clients, companies, engineers, architects, commercial-contacts — second montage, google-calendar, projetactions, metabase).

Absence confirmée : aucune spécification API formelle (OpenAPI/Swagger) n'existe dans le dépôt.

Chaque route gère individuellement l'authentification (`authRequired`), l'autorisation (middleware spécialisé optionnel) et la réponse d'erreur — il n'existe pas de contrat de réponse JSON unifié à l'échelle du projet (les formats `{success, data}`, `{message}`, ou l'objet brut varient selon le module écrit).

10. Authentication

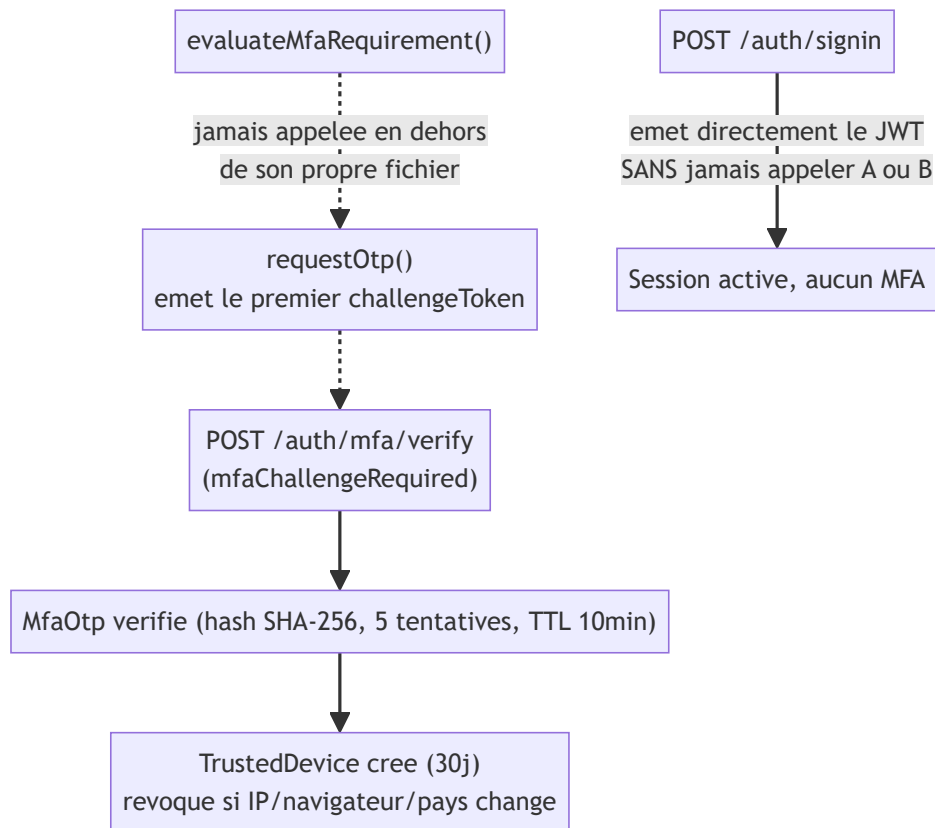


src/middleware/auth.middleware.js::authRequired decode le token, place req.user = {sub, email, role} . Deux secrets JWT distincts : JWT_ACCESS_SECRET (15 min) et JWT_REFRESH_SECRET (7 jours), issuer/audience fixes. mustEnv() fait planter le process au démarrage si l'un manque.

Route	Rôle
POST /auth/signup	Création de compte, rôle auto-assigné par domaine email, isActive:false par défaut (activation admin requise)
POST /auth/signin	Vérification bcrypt + émission immédiate access+refresh token (cookie httpOnly)
POST /auth/refresh	Vérifie le refresh token (cookie ou body) contre le hash bcrypt stocké en base, réemet un access token — pas de rotation du refresh token
POST /auth/logout	Invalide refreshTokenHash en base + clearCookie
POST /auth/forgot-password / reset-password	Token de reset SHA-256 haché, TTL 15 min, rate-limit 10/h/IP + 3/h/email, réponse volontairement neutre (anti-énumération)

11. MFA

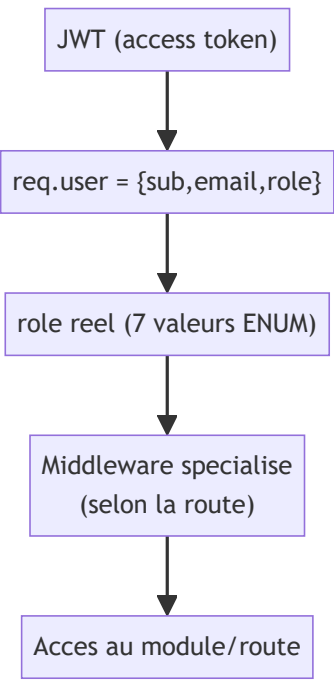
PARTIELLEMENT IMPLÉMENTÉ — Infrastructure MFA complète et fonctionnelle au niveau du code (`src/services/mfa.service.js` , 610 lignes), mais **jamais déclenchée par une connexion réelle**. Détail ci-dessous.



Ce qui existe : OTP 6 chiffres haché SHA-256 (jamais stocké en clair), TTL 10 minutes, 5 tentatives maximum, cooldown de renvoi 60s, ré-vérification périodique adaptée au rôle (3 jours standard, 24h pour admin-tier), détection de changement de contexte (IP/navigateur/pays), appareils de confiance (`TrustedDevice` , 30 jours, révoqués automatiquement si le contexte change), journal complet des tentatives (`MfaAttemptLog`), interrupteur global `MFA_ENABLED` .

Ce qui manque : les fonctions `evaluateMfaRequirement()` et `requestOtp()` — qui décideraient de déclencher un MFA à la connexion et émettraient le premier `challengeToken` — **ne sont appelées par aucun autre fichier du projet** (confirmé par recherche exhaustive). Le code source le documente lui-même : commentaire de `auth.routes.js` — « MFA (conservé pour `/auth/mfa/verify` et `/auth/mfa/resent` uniquement — plus utilisé par `/auth/signin`) ». Conséquence : `/auth/signin` émet le JWT immédiatement, sans jamais passer par un challenge MFA.

12. Autorisation / RBAC



Inventaire complet de `src/middleware/` (17 fichiers)

Middleware	Rôle	Portée
<code>auth.middleware.js</code>	<code>authRequired</code> — décode JWT access	Quasi toutes les routes protégées
<code>moduleAccessGuard.js</code>	Restreint <code>responsable_logistique_achat</code> à une liste de préfixes fixes	Global (<code>app.js:126</code>)
<code>requireRole.js</code>	Factory générique <code>requireRole(...roles) → 403</code>	<code>client.routes.js</code>
<code>requireRootAdmin.js</code>	Réserve l'accès à un seul email en dur	<code>admin.js</code> , <code>archive-requests</code>
<code>requireHrReviewer.js</code>	Admin-tier + email RH dédié	<code>hr-requests</code>
<code>requireMaintenanceManager.js</code>	Admin-tier + <code>responsable_logistique_achat</code> + email dédié	<code>maintenance-requests</code>
<code>kpiRoleGuard.js</code>	Bloque les KPI globaux aux non-admins	<code>dashboard</code> , <code>crm routes</code>
<code>mfaChallenge.middleware.js</code>	Vérifie le <code>challengeToken</code> MFA	<code>auth.routes.js</code> (<code>mfa/verify</code> , <code>resend</code>)
<code>verifyToken.js</code>	Second middleware JWT parallèle (<code>JWT_SECRET</code> , distinct de <code>JWT_ACCESS_SECRET</code>)	Uniquement <code>routes/metabase.js</code>
4 middlewares upload dédiés	Multer par domaine (<code>action/HR/maintenance/POR PROMESH</code>)	Routes correspondantes

Point technique : `verifyToken.js` constitue un second mécanisme d'authentification JWT, non unifié avec `authRequired` , utilisant une variable d'environnement différente (`JWT_SECRET`) — voir section 23.

Erreurs 401 / 403 — quand et pourquoi

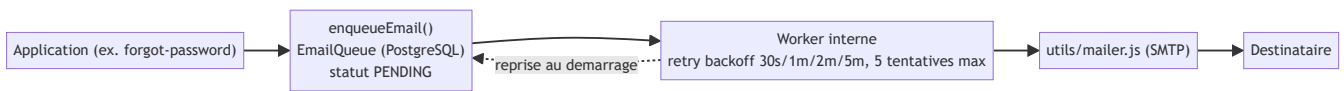
Code	Déclencheur réel
401	<code>authRequired</code> : en-tête manquant ou token invalide/expiré
403	<code>moduleAccessGuard</code> (rôle hors préfixes autorisés), <code>kpiRoleGuard</code> (non-admin demandant un KPI global), <code>requireRootAdmin</code> / <code>requireHrReviewer</code> / <code>requireMaintenanceManager</code> (email/rôle non éligible), vérifications de propriété inline (<code>row.createdBy !== req.user.sub</code>)

13. Gestion des erreurs

Aucun middleware d'erreur Express centralisé (4 arguments) n'existe dans tout le projet — confirmé par recherche exhaustive. Chaque route gère ses erreurs individuellement via try/catch local.

Code	Cause	Origine backend
400	Validation échouée	Joi (modules récents) ou tableaux d'erreurs manuels (routes legacy)
401	Authentification manquante/invalid	<code>authRequired</code> , <code>signin</code> , <code>mfaChallengeRequired</code>
403	Autorisation refusée	Middlewares RBAC (section 12)
404	Ressource introuvable	<code>findByPk</code> renvoyant <code>null</code> , ou route MFA désactivée
409	Conflit	Ex. « Project name already exists », « Email already used »
422	Non identifié dans le code source analysé	Aucune occurrence de <code>res.status(422)</code> trouvée
429	Rate limiting	<code>express-rate-limit</code> (routes auth), cooldown MFA
500	Erreur générique	Catch-all — <code> </code> au moins un endpoint (<code>commercial contacts.routes.js</code>) renvoie le <code>stack</code> d'erreur brut dans la réponse JSON — fuite d'information potentielle

14. Email



Trois implémentations email coexistent — redondance confirmée, pas totalement du code mort :

Fichier	Rôle réel	Consommateurs
utils/mailer.js	SMTP le plus soigné (vérification de connexion, enveloppe explicite)	Uniquement emailQueue.service.js
services/email.service.js	Envoi SMTP direct, sans file d'attente ni retry (commentaire du code : « ⚠ dev only » pour tls.rejectUnauthorized:false)	Crons, notifications, plusieurs modules
services/emailService.js	Transporteur codé en dur pour Gmail, export unique sendRelanceEmail	Bug confirmé : notifications.routes.is appelle emailService.send(...) . méthode inexistante — TypeError systématique sur POST /notifications/send-email , documenté par le code source lui-même

File d'attente (emailQueue.service.js) : modèle EmailQueue , statuts PENDING/RETRYING/SENT/FAILED/CANCELLED , ligne créée en base avant toute tentative (jamais perdue en cas de crash), retry uniquement sur erreurs SMTP temporaires (codes 451/429), reprise automatique au démarrage. enqueueEmail() ne rejette jamais — garantit que l'appelant n'échoue pas pour une raison SMTP.

15. Notifications

Modèle `Notification` : colonnes flexibles nullable (`projectId`, `commentId`, `commercialContactId`, `relanceId`, `maintenanceRequestId`, `hrRequestId`, `actionId`) sans FK stricte — relation "à tout" via colonnes optionnelles. Indexes de performance ajoutés suite à un incident réel de volumétrie (section 23).

Service : `listMyNotifications` (paginé), `markAllRead`, `markOneRead`, `deleteNotification`, `sendEmailNotification`. Routes : `GET /`, `POST /send-email` (bug `TypeError`, section 14), `PUT /read-all`, `PUT /:id/read`, `DELETE /:id`. Complément temps réel via `Socket.IO` (`emitToUser()`) utilisé en parallèle par les crons de rappel.

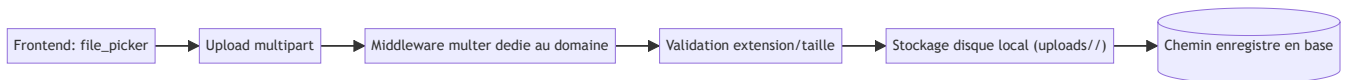
16. Upload / Documents

Fichier mort confirmé : `src/routes/upload.js` — nom de fichier contenant un espace littéral, jamais monté dans `app.js` ni ailleurs, et syntaxiquement invalide en isolation (référence `router` non déclarée). Code orphelin.

Redondance confirmée : `middleware/upload.js` et `middleware/upload.middleware.js` sont deux implémentations multer quasi identiques (avatars, 3 Mo, whitelist d'extensions) — seule la seconde a un consommateur vivant (`users.routes.js`).

Middlewares upload additionnels par domaine : `middleware/uploads.js` (pièces jointes commerciales), `actionUpload.middleware.js`, `hrRequestUpload.middleware.js`, `maintenanceRequestUpload.middleware.js`, `porPromeshAttachment.middleware.js`.

Stockage : exclusivement local disque (`multer.diskStorage`) — aucun stockage cloud (S3/GCS/Azure Blob) identifié . Servi statiquement via `app.use("/uploads", express.static("uploads"))` . Pas d'authentification sur cette route statique — tout fichier dont le chemin est deviné est accessible sans JWT (signalé aussi dans `docs/audit/Technical_Debt.md` comme dette ouverte).

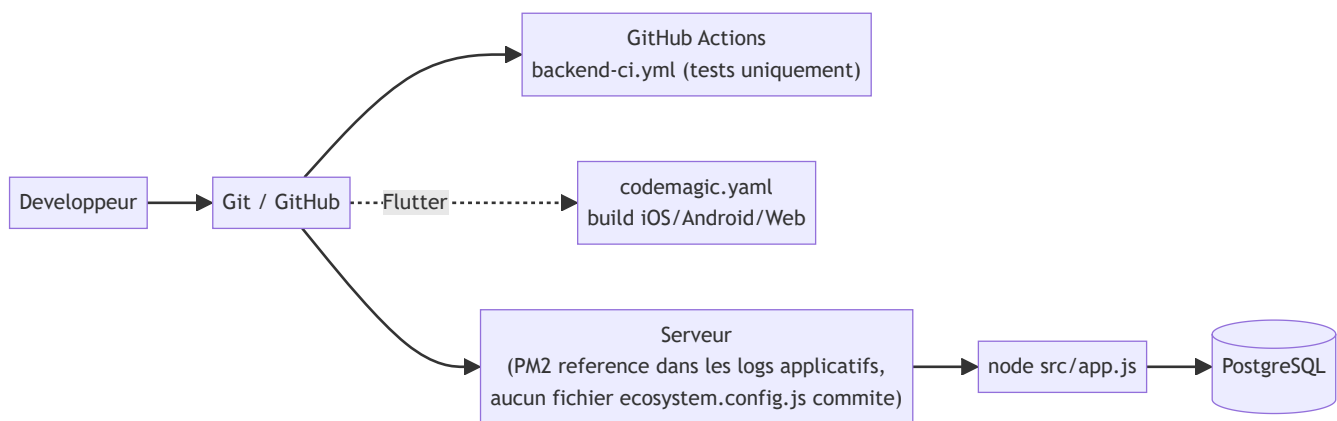


Aucun pipeline OCR (extraction de texte depuis un document scanné) n'a été identifié dans le code source analysé.

17. Cron / Scheduler

Détaillé section 7. Rappel synthétique : 5 jobs `node-cron` , tous protégés par un interrupteur `AUTO_EMAIL_JOBS_ENABLED` (sauf le monitoring IA proactif, interrupteur séparé). Le job le plus sensible historiquement (`followup.job.js`) a produit un incident réel de volumétrie (12M+ lignes de notification en 80 jours, voir section 23) avant correctif de déduplication.

18. Déploiement, DevOps, PM2, Environnements



CI/CD réel

Confirmé : `.github/workflows/backend-ci.yml` — exécute les tests Jest contre un conteneur PostgreSQL 17 de service, lance les migrations (`continue-on-error` , dette suivie séparément), `npm audit` non bloquant. `codemagic.yaml` (racine Flutter) — build IPA iOS release uniquement, pas de suite de tests Flutter dans ce pipeline.

PM2

PARTIEL — Aucun fichier `ecosystem.config.js` n'est commité dans le dépôt. PM2 est néanmoins référencé comme outil d'exploitation réel dans plusieurs commentaires du code applicatif (logs explicitement conçus pour être « grep-ables dans `pm2 logs` »), ce qui indique un usage opérationnel non versionné plutôt qu'une configuration Infrastructure-as-Code.

Docker

Aucun `Dockerfile` ni `docker-compose.yml` n'a été trouvé dans le dépôt.

Environnements

Un seul fichier `.env.example` existe (pas de distinction explicite `.env.development` / `.env.test` / `.env.production` dans le dépôt, si ce n'est les variables d'environnement injectées directement par le workflow CI pour l'environnement de test). Variables réelles (valeurs jamais affichées) :

Catégorie	Variables
Serveur	PORT, APP_URL
Base de données	DATABASE_URL, DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD
JWT	JWT_ACCESS_SECRET=[SECRET], JWT_REFRESH_SECRET=[SECRET], JWT_MFA_SECRET=[SECRET], ACCESS_TOKEN_EXPIRES_IN, REFRESH_TOKEN_EXPIRES_IN
MFA	MFA_ENABLED
Email	EMAIL_HOST, EMAIL_PORT, EMAIL_USER=[SECRET], EMAIL_PASS=[SECRET], EMAIL_SECURE, EMAIL_FROM
Google Calendar	GOOGLE_CLIENT_ID=[SECRET], GOOGLE_CLIENT_SECRET=[SECRET], GOOGLE_REDIRECT_URI, GOOGLE_TOKEN_ENC_KEY=[SECRET], GOOGLE_CALENDAR_WEBHOOK_BASE_URL
Comptes superadmin	SUPERADMIN_EMAIL=[SECRET], SUPERADMIN_PASSWORD=[SECRET], SUPERADMIN_1_EMAIL=[SECRET], SUPERADMIN_1_PASSWORD=[SECRET]
Divers	RECAPTCHA_SECRET=[SECRET], AUTO_EMAIL_JOBS_ENABLED
IA (Document 3)	AI_COPILOT_ENABLED, AI_COPILOT_ROLES_ALLOWED, AI_LLM_PROVIDER, AI_LLM_MODEL, AI_LLM_API_KEY=[SECRET], OLLAMA_BASE_URL, OLLAMA_MODEL, OLLAMA_TIMEOUT_MS, AI_PROACTIVE_MONITORING_ENABLED

19. Logging / Monitoring

Un module de logging structuré existe (`src/utls/logger.js` , namespaces — utilisé systématiquement dans le kernel IA). **Partiellement adopté** : le CRM historique utilise majoritairement `console.log` / `console.error` directement plutôt que ce logger (75 fichiers concernés, selon l'audit préexistant `docs/audit/Technical_Debt.md` , cohérent avec les motifs `console.error` observés dans les services lus pour ce rapport, ex. `commercialContactRelance.service.js`).

Aucun outil d'observabilité applicative (APM, métriques, tracing distribué) n'a été identifié dans le code source analysé. Les logs sont consultables via la sortie standard du process (donc via `pm2 logs` en exploitation réelle, d'après les références de code déjà notées section 18), pas via une plateforme de logs centralisée (ELK, Datadog, etc.).

20. Responsive Web / Tablet / Mobile

`responsive_framework` (5 breakpoints : XS 0-576, SM 577-904, MD 905-1239, LG 1240-1439, XL 1440+) piloté globalement dans `main.dart`. Dans `AppShell`, le seuil `>1240px` détermine sidebar fixe (desktop) vs `Drawer` (mobile/tablette) ; `ResponsiveRowColumn` / `ResponsiveValue` adaptent la disposition et les paddings. 86 fichiers complètent ce mécanisme structurel par des ajustements ponctuels (`MediaQuery` / `LayoutBuilder`), notamment dans les grilles de modules PROBAR/POR PROMESH et les écrans RH.

Aucun problème de *RenderFlex overflow* ou de débordement spécifique n'a pu être confirmé par lecture statique du code dans le périmètre de cet audit (ce type de défaut se constate typiquement à l'exécution, pas par lecture de code) — à vérifier par un test manuel multi-résolution si nécessaire.

21. Sécurité

Une documentation de sécurité préexistante et détaillée existe déjà (`docs/security/Security_Report.md` , `Threat_Model.md` , `Security_Fixes.md` , `Security_Test_Plan.md`) — non intégralement relue dans cette passe, mais cohérente avec les constats indépendants suivants, obtenus par lecture directe du code pour ce rapport :

Constat	Statut
helmet + CORS whitelist d'origines en dur + cookies httpOnly pour le refresh token	Confirmé
Mots de passe hachés bcrypt, tokens de reset hachés SHA-256, OTP MFA hachés SHA-256	Confirmé
Anti-énumération d'email sur forgot-password (réponse neutre systématique)	Confirmé
Deux mécanismes JWT parallèles non unifiés (<code>authRequired</code> vs <code>verifyToken.js</code>)	Point technique
Fuite de stack trace dans une réponse d'erreur API (<code>commercial_contacts.routes.js</code>)	Point technique
Route statique <code>/uploads</code> sans authentification	Dette ouverte (également notée dans l'audit préexistant)
MFA jamais déclenché en pratique (section 11)	Dette ouverte
Sécurité du kernel IA (RBAC, ownership, prompt injection, MCP)	Auditée en détail — voir Document 3

22. Points forts

- **Séparation frontend/backend nette**, API REST/WebSocket claire entre les deux applications.
- **Kernel IA en Ports & Adapters** rigoureux, vérifié mécaniquement par des tests "policy" qui lisent le code source pour garantir le respect des frontières (voir Document 3).
- **Modules récents en couches** (`src/modules/*`) avec validation Joi dédiée, séparation repository/service/controller réelle.
- **File d'attente email robuste** (jamais de perte, retry avec backoff, reprise au démarrage).
- **Migrations disciplinées** : jamais de `sync()` automatique, historique complet via `sequelize-cli`.
- **MFA** : bien que non branché, l'infrastructure elle-même (hachage, TTL, anti-rejeu, détection de contexte, appareils de confiance) est complète et de bonne qualité.
- **RBAC multi-niveaux** : middleware global + middlewares spécialisés par domaine métier, cohérent avec les rôles réels de l'ENUM.
- **CI backend existante** (même minimale) et pipeline de build Flutter (`codemagic.yaml`).

23. Problèmes techniques identifiés

Problème	Localisation	Impact	Cause	Solution	Priorité
MFA jamais déclenché à la connexion	auth.routes.js (signin)	Aucune 2FA effective en production malgré l'infrastructure complète	<code>evaluateMfaRequirement()</code> / <code>requestOtp()</code> jamais appelées	Brancher ces fonctions dans le flux <code>signin</code>	HIGH
3 implémentations email redondantes, dont une cassée	utils/mailler.js, services/email.service.js, services/emailService.js	POST <code>/notifications/send-email</code> échoue systématiquement (TypeError)	Duplication historique, méthode inexistante appelée	Unifier sur <code>email.service.js</code> + file d'attente, supprimer <code>emailService.js</code>	MEDIUM
Deux middlewares JWT parallèles	auth.middleware.js vs verifyToken.js	Confusion de maintenance, risque de mauvaise configuration (<code>JWT_SECRET</code> non défini)	Évolution historique non unifiée	Migrer <code>metabase.js</code> vers <code>authRequired</code> , supprimer <code>verifyToken.js</code>	MEDIUM
Fichier route mort et syntaxiquement invalide	<code>src/routes/upload.js</code> (espace dans le nom)	Aucun (jamais chargé), mais confusion de code	Fichier orphelin non nettoyé	Suppression	LOW
Redondance <code>upload.js</code> / <code>upload.middleware.js</code>	<code>src/middleware/</code>	Maintenance dupliquée	Deux implémentations quasi identiques	Supprimer la version non consommée	LOW
Aucun middleware d'erreur centralisé	<code>app.js</code> (absence)	Formats d'erreur incohérents, fuite de stack trace possible	Gestion d'erreur locale à chaque route depuis l'origine du projet	Ajouter un middleware d'erreur global (4 arguments), retirer les <code>e.stack</code> des réponses	HIGH
Route statique <code>/uploads</code> non authentifiée	<code>app.js</code> (<code>express.static</code>)	Tout fichier au chemin deviné est accessible sans JWT	Middleware d'authentification non appliqué à cette route	Servir les fichiers via une route authentifiée dédiée	HIGH
Double montage de routes (<code>/projects</code> , <code>/utils</code> , <code>/commercial-contacts</code>)	<code>app.js</code>	Comportement dépendant de l'ordre interne des routers, risque de collision silencieuse	Coexistence legacy/modules en couches sur le même préfixe	Fusionner ou distinguer les préfixes	MEDIUM
Trois strates architecturales non unifiées	<code>src/routes/</code> vs <code>src/controllers/</code> vs <code>src/modules/</code>	Coût d'onboarding, incohérence de style	Évolution progressive du projet sans refactor rétroactif	Convention unique pour les nouveaux modules (déjà le cas pour les modules récents)	MEDIUM
Aucun linter/formatter (ESLint/Prettier)	Projet entier (backend)	Style de code non uniformisé	Non installé	Ajouter ESLint + Prettier + hook pre-commit	MEDIUM
<code>console.log</code> direct dans 75 fichiers plutôt que le logger structuré	CRM historique	Logs non filtrables par namespace/niveau	Logger introduit après une grande partie du code	Migration progressive vers <code>utils/logger.js</code>	LOW
Aucune spécification OpenAPI/Swagger	Projet entier	Documentation API non générée, onboarding plus lent	Non mis en place	Générer un contrat OpenAPI a minima pour les endpoints critiques	LOW
Absence de Docker	Projet entier	Déploiement non reproductible/portable	Non mis en place	Conteneuriser backend + PostgreSQL de dev	MEDIUM
Incident historique de volumétrie notifications (12M+ lignes)	<code>cron/followup.job.js</code>	Dégradation de performance en production (corrigé)	Absence de déduplication	Déduplication ajoutée + index de performance	LOW (corrigé)

Ce tableau combine des constats obtenus par lecture directe du code (première moitié) et des éléments corroborés par la documentation d'audit préexistante [docs/audit/Technical_Debt.md](#) (incident de volumétrie, absence de linter, `console.log`). Chaque ligne reste vérifiable dans le code cité.

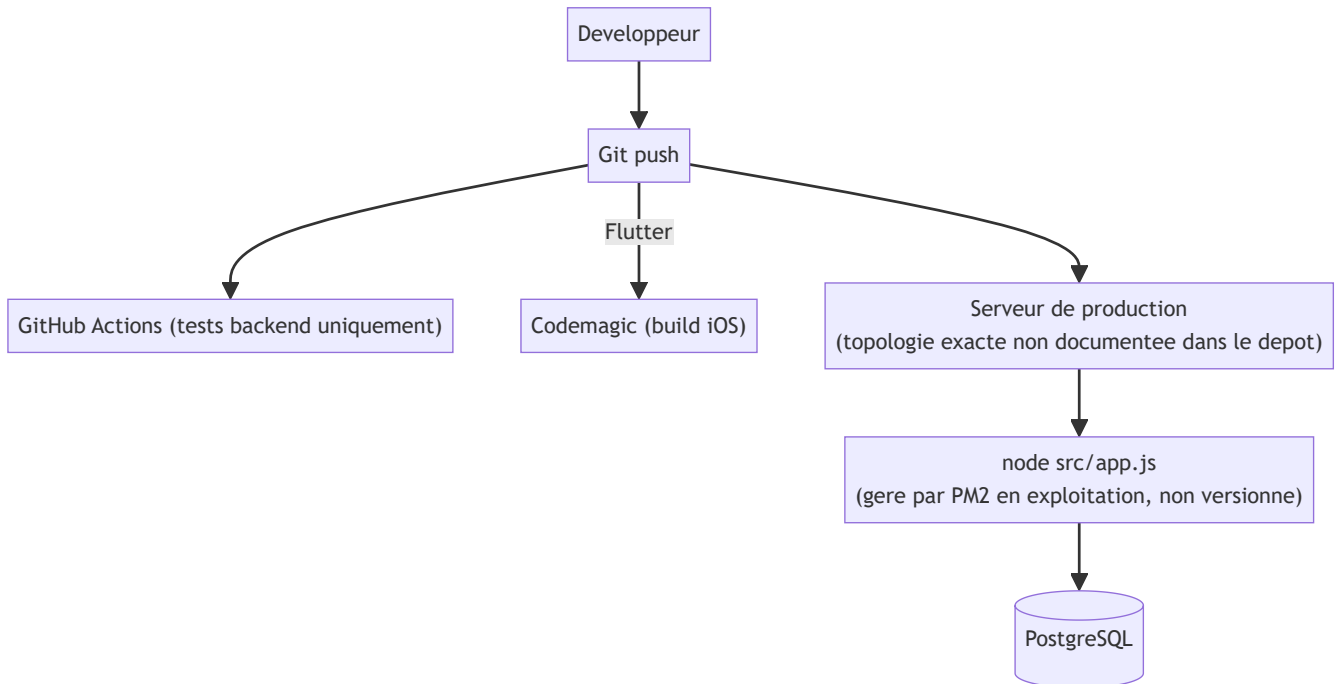
24. Architecture cible (recommandations)

Cette section est une proposition, distincte de l'architecture actuelle décrite précédemment — aucun des éléments ci-dessous n'existe dans le code au moment de cet audit.

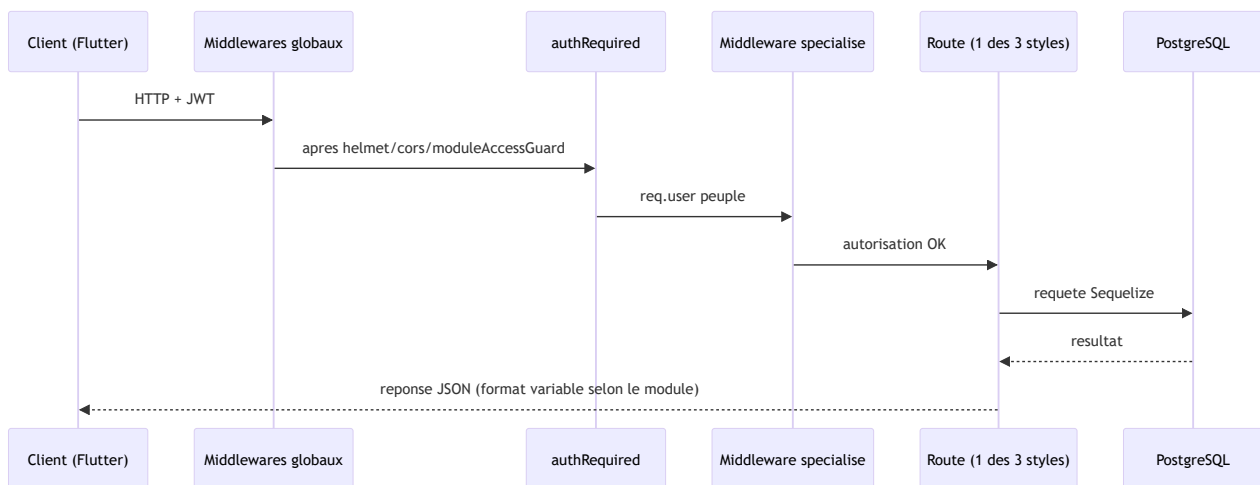
Axe	Recommandation
Erreurs	Middleware d'erreur Express centralisé, format de réponse JSON unifié projet-wide
Authentification	Brancher le flux MFA existant sur <code>/auth/signin</code> , unifier les deux mécanismes JWT
Fichiers	Route authentifiée pour servir les uploads, ou stockage objet (S3-compatible) avec URLs signées
Architecture backend	Généraliser le pattern "module en couches + Joi" aux routes legacy restantes
DevOps	Conteneurisation Docker, fichier <code>ecosystem.config.js</code> PM2 versionné, CI étendue au frontend
Observabilité	Migration complète vers le logger structuré, ajout d'un APM/tracing
Documentation API	Spécification OpenAPI générée à partir des validators Joi existants
Qualité de code	ESLint + Prettier avec hook pre-commit

25. Diagrammes complémentaires

Architecture de déploiement (état réel)



Cycle de vie d'une requête (vue unifiée)



26. Conclusion

L'architecture technique du CRM PROBAR présente un contraste net entre un kernel IA construit avec une rigueur d'ingénierie logicielle moderne (Ports & Adapters, tests de garantie architecturale, documenté en détail dans le Document 3) et un CRM historique fonctionnel mais architecturalement hétérogène, résultat d'une évolution progressive sur trois générations de conventions distinctes. Les points techniques les plus significatifs identifiés dans ce document — un système MFA complet mais débranché du flux de connexion réel, l'absence de gestion d'erreur centralisée, une route de stockage de fichiers non authentifiée, et une triple implémentation email dont une est cassée — sont tous vérifiables directement dans le code cité, et aucun n'a été supposé à partir de pratiques usuelles de CRM.

Cette base reste néanmoins fonctionnelle et démontrable : l'authentification JWT, le RBAC multi-niveaux, la file d'attente email, et la discipline de migration de schéma constituent des fondations saines sur lesquelles les corrections identifiées (section 23) et l'architecture cible (section 24) peuvent s'appuyer sans réécriture complète.

Les architectures métier (modules, workflows, rôles, permissions, user journeys), base de données/API détaillées, et Copilot IA font l'objet des Documents 2, 4 et 3 respectivement.

Document 1/5 — CRM_Architecture_Complete — Projet PROBAR — 29 août 2026

Fondé exclusivement sur la lecture directe du code source et une documentation d'audit préexistante, systématiquement recoupée.